CSCI 599 Assignment 2 — Multi-Agent System with RAG and Orchestration

Fall 2026. Individual assignment. 15% of course grade. Due date: see Brightspace.

---

1. Objective

Build and deploy a multi-agent system where three or more specialized agents cooperate under supervisor or hierarchical orchestration, retrieve grounded context through a RAG pipeline over a document corpus of your choice, and route work across at least two different LLM providers based on cost-vs-capability tradeoffs. The deliverable runs on Google Cloud Run — a FastAPI backend serves a Server-Sent Events token stream to a vanilla HTML/JS chat interface with inspectable agent traces.

Complexity floor: equal to or greater than Assignment 1. Distinct architectural pattern — Assignment 1 taught a single-agent MCP tool loop; Assignment 2 teaches multi-agent orchestration with retrieval.

---

2. Description

You build a **FastAPI application** that dispatches a user query to a **supervisor** or **hierarchical** orchestrator implemented as a LangGraph state graph. The orchestrator delegates work to at least three specialized agents. At least one agent uses **RAG** over a Chroma or Pinecone vector store containing a document corpus you select. At least one agent uses **real-time knowledge retrieval** via Tavily Remote MCP, or an approved externally updated domain-specific alternative. The orchestrator routes generation calls across at least **two different LLM providers** with an inspectable routing decision. Response tokens stream to the browser using Server-Sent Events; the frontend is a single vanilla HTML/CSS/JS page served by FastAPI as static content.

2.1 Agent specialization floor

At least three specialized agents from the syllabus menu:

- **Research Agent** — web search plus RAG retrieval, returns grounded answers with citations
- **Code Agent** — code generation or code analysis over a supplied file or corpus
- **Data Agent** — structured data queries (SQLite, CSV, or a JSON document store)
- **Summary Agent** — document summarization with citation preservation

- **Router / Supervisor Agent** — orchestration, not counted toward the three specialized agents

Specialization is defined by **distinct tool access and distinct system prompts**, not by cosmetic labels on the same underlying LLM call. An agent that has no unique tool access and no unique prompt is a wrapper, not an agent.

2.2 Orchestration pattern

Pick **one**:

Pattern	State topology	Agent addressability	Typical failure mode
Supervisor	Central supervisor node routes to workers; workers return to supervisor	Flat: supervisor knows all workers	Supervisor becomes a bottleneck for complex plans
Hierarchical	Team supervisors coordinate worker teams; top-level supervisor coordinates team supervisors	Nested: top supervisor sees teams, not individual workers	Cross-team coordination requires escalation
Codex worktree-based multi-agent	Parallel agents operate in isolated worktrees and coordinate through an explicit task/delegation structure	Worktree/task based	Integration and coordination across parallel branches require explicit handling

The Supervisor and Hierarchical patterns are implemented in LangGraph. The syllabus also permits **Codex's worktree-based multi-agent model** as an alternative. The reference architecture guidance below assumes LangGraph, but the Codex path is graded against equivalent evidence of genuine multi-agent delegation, coordination, inspectability, and failure handling.

2.3 LLM routing floor

At least **two different LLM providers** wired into the system, with routing that is **inspectable and rationalized**. Two models from the same provider do not satisfy this requirement.

Acceptable routing dimensions:

- **Cost-based** — cheaper model for classification and short lookups; more capable model for multi-step synthesis

- **Capability-based** — smaller model for retrieval-augmented lookups; larger model for long-context reasoning
- **Hybrid** — combines both dimensions

The routing decision must be visible in the UI or in the trace log. If the grader cannot tell why provider A got the query and provider B did not, the routing is not intelligent — it is hard-coded.

> **Papa's rule:** A router that always picks the same model is not a router. It is a hard-coded call with extra steps.

2.4 RAG pipeline floor

You ship a working RAG pipeline over a document corpus of your choice. The corpus must have genuine retrieval structure — course readings, research papers, technical documentation, an organization's public docs. A five-file toy corpus does not demonstrate mastery.

Required pipeline stages:

1. **Ingestion** — load documents, chunk them, produce embeddings
2. **Storage** — write chunks and embeddings to Pinecone (named/default syllabus option) or Chroma (equivalent supported alternative)
3. **Retrieval** — retrieve top-k chunks for a query
4. **Augmentation** — inject retrieved chunks into the generation prompt
5. **Generation** — produce the answer with citations back to source chunks

Evaluation via RAGAS — faithfulness, answer relevance, context precision, context recall — over a fixed test question set you define. RAGAS results ship in `docs/evaluation.md`.

2.5 Real-time knowledge integration

At least one agent queries **Tavily Remote MCP** for current-events grounding. This is the same Remote MCP pattern used in Assignment 1 — Streamable HTTP transport, API key passed as a query parameter to the Remote MCP endpoint. Rationale: RAG over a static corpus is a search engine. RAG combined with a live web-retrieval channel is an agent.

Vertex AI Search is permitted as an alternative only when the project uses an externally updated domain-specific source that provides the live/current knowledge required by this criterion. A static Vertex AI Search datastore over the same fixed corpus used for RAG does not substitute for the real-time knowledge requirement. See §3.4.

2.6 Frontend

FastAPI backend serving Server-Sent Events (SSE) token streams to a vanilla HTML/CSS/JS frontend. At minimum:

- Chat UI with query input and a streaming response area that consumes an SSE/token-streaming endpoint
- Visible routing decision per turn (which agent was invoked, which LLM provider was routed to, why)
- Inspectable trace log showing orchestration state as it moved through the system
- Citations for RAG-retrieved content, linked to source document identifiers

The frontend is a single HTML document with vanilla JavaScript consuming the streamed response. The client implementation may use `fetch()` with a streaming response reader or another standards-compatible SSE consumption pattern appropriate to the request method. No frontend framework is required — no React, Vue, Angular, or Svelte build step, no `node_modules` in the container image. FastAPI serves the HTML, CSS, and JS files as static content alongside the API routes.

Alternative permitted: FastAPI + HTMX. HTMX is a small JavaScript library that adds `hx-*` attributes to HTML elements for server-driven interactivity — no WebSocket dependency, works cleanly with Cloud Run + FastAPI + SSE. HTMX-based implementations satisfy all four requirements above and are graded identically to vanilla HTML/JS implementations. Students choosing HTMX should include the `htmx.min.js` script in the same static-assets folder and document the framework choice in the README.

2.7 Deployment

Google Cloud Run. Container ships the FastAPI app, orchestration layer, RAG index or connection, the vanilla HTML/JS frontend as static assets, and provider SDKs. Localhost-only submissions are not acceptable per syllabus (§Assignment Submission Policy). See §4 for the deployment recipe.

If the application claims multi-turn state that must survive Cloud Run instance recycling, restarts, or scaling, that state must use **durable external persistence**. Supported approaches include **Firestore, Cloud SQL, or Google Cloud Storage (GCS) mounted through Cloud Storage FUSE/GCSFuse**. A genuinely stateless or single-turn design does not need a durable conversation-state store.

---

3. APIs, LLMs, and configuration

3.1 Framework stack

Component	Framework	Attribution
Orchestration	LangGraph; Codex worktree-based multi-agent model also permitted	LangChain / OpenAI
RAG pipeline	LangChain LCEL	LangChain

Component	Framework	Attribution
Backend / streaming	FastAPI + SSE	Sebastián Ramírez (FastAPI) / W3C HTML Living Standard (SSE)
Frontend	Vanilla HTML/CSS/JS	(no framework)
Frontend (alternative)	HTMX	Big Sky Software (`htmx.org`)
Vector store (syllabus named/default)	Pinecone	Pinecone Systems Inc.
Vector store (equivalent alternative)	Chroma, embedded	Chroma Inc.
Evaluation	RAGAS	Exploding Gradients
Real-time knowledge	Tavily Remote MCP	Tavily

Pin every framework in `requirements.txt` with an exact version. `>=` and `~=` produce "worked on my machine" drift between your submission and TA regrade.

Students may vibe-code the HTML/CSS/JS frontend using Codex CLI or Claude Code from a detailed prompt, following the Week 13 Lecture 26 pattern. The frontend is a single-file HTML document with vanilla JavaScript consuming the streamed endpoint — well within a single vibe-coding iteration. Bundling the vibe-coded frontend with the FastAPI backend as static assets served from a static route is the canonical deployment shape.

3.2 LLM providers

You wire at least **two different providers** into the routing layer. Three providers are recommended for a meaningful shootout — cost, latency, and answer quality vary in ways that only become visible across a spread.

Current provider examples for this assignment:

Provider	Model reference	Access	Primary use in the shootout
OpenAI	GPT-6 Sol	OpenAI API key	Frontier reasoning/synthesis baseline
Anthropic	Claude Sonnet (current release)	Anthropic API key	Long-context synthesis and structured extraction
Google	Gemini 3.8 Flash	Google API key	Fast, cost-sensitive generation; independent provider

Provider	Model reference	Access	Primary use in the shootout
NVIDIA	Nemotron 3 Ultra / current NIM model	NVIDIA API key	Free/student-accessible provider option where available

NVIDIA NIM is a provider option, not a substitute for the two-provider requirement. A student may use NVIDIA as one of the required providers, but a system that routes only among NVIDIA-hosted models does not satisfy the requirement. Likewise, routing between two OpenAI models does not satisfy it.

Verify and pin current model IDs. Provider catalogs rotate faster than the assignment production cycle. The table above identifies the current model families used for course planning; your `.env.example` and source code must contain the exact deployable model IDs you verified when you submit.

Cost-access asymmetry. Students are not required to choose the same provider pair as the reference implementation. Choose two or more providers you can access and document your routing logic. NVIDIA NIM may provide a lower-cost/free path for one side of that pair.

Exact commercial pricing and free-tier terms change. Verify current pricing on each provider's pricing page when you plan your budget. USC students receive Google Cloud credit that covers eligible Cloud Run costs; LLM API costs may bill separately depending on provider and account.

> **Papa's rule:** Cost-access asymmetry is not a footnote. Two students with the same technical skill and different credit cards should get the same grade. Provider choice is flexible; the architectural requirement is genuine routing across at least two different providers.

3.3 Embedding model

OpenAI `text-embedding-3-small` is the reference implementation's corpus embedding model. Local embedding via `sentence-transformers` is permitted for students who want zero external-service dependence for embeddings — the tradeoff is embedding latency during corpus ingestion and potentially different vector dimensionality.

Embedding choice does not count toward the two-generation-provider requirement.

3.4 Real-time knowledge integration

Tavily Remote MCP. Hosted MCP server for web search and retrieval. Same Remote MCP pattern established in Assignment 1 — no local MCP server process and no OAuth flow. The API key is passed as a query parameter to the Remote MCP endpoint.

Endpoint pattern:

`https://mcp.tavily.com/mcp/?tavilyApiKey=<your-tavily-key>`

Transport: Streamable HTTP. Your agent can use `langchain-mcp-adapters` to connect to the Remote MCP URL and expose Tavily's search and extract tools to agents that need live web retrieval.

Setup:

1. Sign in at `tavily.com` (create an account if you don't have one from Assignment 1)
2. Claim the CSCI 599 student/free plan if available
3. Copy the API key from your Tavily dashboard
4. Pass the key via the query parameter shown above; do not put it in the request body

The Tavily student/free plan is currently available and is the recommended low-cost path. **Quota and plan terms are volatile and may be updated during the semester; verify the current terms when you set up the assignment.**

Alternative: Vertex AI Search. Vertex AI Search may substitute for Tavily when the student's project uses an **externally updated domain-specific source** that supplies current/live information appropriate to the project. The tradeoff is a heavier GCP setup path (datastore provisioning, service account, IAM configuration) in exchange for domain-scoped retrieval. A static Vertex AI Search datastore containing only the same fixed corpus used by the RAG pipeline does not satisfy the separate real-time knowledge requirement. Students choosing the qualifying Vertex AI Search path are not graded down.

3.5 Vector store

Pinecone is the named/default vector store in the syllabus. Chroma is an equivalent supported alternative for this assignment. Students are graded identically on either path.

The reference implementation uses **Chroma embedded** because:

- Embedded — no external vector service to provision
- No vector-store API key
- Runs on a laptop and inside the reference container
- Same retrieval code path across environments

Pseudo-code shape for a Chroma ingestion pipeline:

```
# Pseudo-code — not a reference implementation.
# Your code needs error handling, chunking configuration, and re-indexing log
ic.

from chromadb import PersistentClient
from langchain_openai import OpenAIEmbeddings
```

```
client = PersistentClient(path="./chroma_index")
collection = client.get_or_create_collection(name="corpus")

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

for chunk_id, chunk_text in chunks:
    vector = embeddings.embed_query(chunk_text)
    collection.add(
        ids=[chunk_id],
        embeddings=[vector],
        documents=[chunk_text],
        metadatas=[{"source": chunk_source(chunk_id)}],
    )
```

Retrieval pattern:

```
# Pseudo-code — shape only.

results = collection.query(
    query_embeddings=[embeddings.embed_query(user_query)],
    n_results=5,
)

retrieved_chunks = results["documents"][0]
```

Pinecone path. Students choosing the syllabus-default Pinecone path use a Pinecone index and the LangChain `PineconeVectorStore` retriever pattern. Pinecone account/free-tier terms can change; verify current terms before committing to that path.

3.6 Environment variables

Your `.env` file (not committed to source control) supplies the providers and services you actually use. Example shape:

```
# LLM providers — verify and pin exact current model IDs
OPENAI_API_KEY=sk-...
OPENAI_MODEL=<current-gpt-6-sol-api-model-id>

ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=<current-claude-sonnet-model-id>

GOOGLE_API_KEY=...
GEMINI_MODEL=gemini-3.8-flash

NVIDIA_API_KEY=nvapi-...
NVIDIA_MODEL=<current-nemotron-nim-model-id>
NVIDIA_BASE_URL=https://integrate.api.nvidia.com/v1

# Embedding
```

```
OPENAI_EMBEDDING_MODEL=text-embedding-3-small

# Vector store — configure the path you chose
CHROMA_PERSIST_DIR=./chroma_index

# Real-time knowledge
TAVILY_API_KEY=tvly-...

# Cloud Run deployment
GCP_PROJECT_ID=your-project-id
GCP_REGION=us-west1
CLOUD_RUN_SERVICE=multi-agent-rag

# Durable state — configure the backend you chose when required
STATE_BACKEND=firestore
FIRESTORE_PROJECT_ID=your-project-id

# GCS/GCSFuse alternative
GCS_STATE_BUCKET=your-state-bucket
GCS_STATE_MOUNT=/mnt/state
```

If you use Pinecone, add the Pinecone variables required by your current SDK. If you use Cloud SQL or another approved durable-state backend, document and configure its corresponding variables instead.

Ship a `.env.example` in your repository with placeholder values. Never commit `.env`.

---

4. Cloud Run deployment

4.1 Prerequisites

- `gcloud` CLI installed and authenticated
- GCP project with billing enabled (including applicable USC/Google Education credit)
- Required APIs enabled for the services your deployment uses

Reference baseline:

```
gcloud services enable run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com
```

Enable Firestore, Cloud Storage, Cloud SQL, or other APIs as required by your chosen persistence path.

4.2 Dockerfile shape

Multi-stage build. Python 3.13-slim base. Non-root user. FastAPI runs under `uvicorn` and binds to Cloud Run's injected `PORT` environment variable.

```
# Pseudo-code shape — your Dockerfile needs the full COPY, RUN, and CMD lines.

FROM python:3.13-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.13-slim
WORKDIR /app
COPY --from=build /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
COPY . .
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
CMD ["uvicorn", "main:app", \
     "--host", "0.0.0.0", \
     "--port", "8080"]
```

4.3 FastAPI + SSE on Cloud Run

Standard Cloud Run deployment. FastAPI's ASGI stack over `uvicorn` binds to `PORT` and streams response tokens using SSE. Set an appropriate Cloud Run request timeout for long-running streams. No separate load-balancer configuration is required for the coursework deployment.

The browser client is not required to use native `EventSource`. A POST-based chat endpoint may use `fetch()` with streaming response consumption; a GET-based design may use `EventSource`. What is graded is working token streaming and the required trace/citation surfaces, not the choice between those client APIs.

4.4 State persistence across Cloud Run instances

Cloud Run may recycle, restart, or scale application instances. **In-process memory is not durable state.**

If your design claims conversation or agent state that survives those events, use durable external persistence. Supported coursework approaches include:

- **Firestore** — reference implementation's canonical durable checkpointer path
- **Google Cloud Storage (GCS) with native Cloud Storage FUSE/GCSFuse volume mounts** — valid durable file/object-backed persistence path
- **Cloud SQL** — valid durable database-backed persistence path

- **Equivalent external durable store** — document the choice and demonstrate that it survives instance replacement

Cloud Run Instances may be used. When durable state is required, GCS/GCSFuse or another external durable store remains an accepted persistence mechanism.

A **genuinely stateless or single-turn system** does not need durable conversation persistence. Do not add a database merely to satisfy a requirement your architecture does not claim.

For local development, a LangGraph SQLite checkpointer or in-memory test store is acceptable. Do not claim local-only in-process state as durable Cloud Run persistence.

Chroma persistence is a separate concern — it stores the RAG index, not necessarily per-user conversation state. The reference implementation may bake a read-only Chroma index into the container image; Pinecone users keep the vector index externally.

> **Papa's rule:** If your deployed agent claims memory across sessions or instance replacement, persist it outside process memory. Firestore, GCS/GCSFuse, Cloud SQL, or an equivalent durable store are all valid architectural choices.

4.5 Deploy commands

```
# Pseudo-code shape. Adapt project ID, region, service name,
# environment variables, and persistence configuration.

gcloud config set project ${GCP_PROJECT_ID}

gcloud builds submit --tag gcr.io/${GCP_PROJECT_ID}/multi-agent-rag

gcloud run deploy multi-agent-rag \
    --image gcr.io/${GCP_PROJECT_ID}/multi-agent-rag \
    --platform managed \
    --region ${GCP_REGION} \
    --allow-unauthenticated \
    --memory 1Gi \
    --cpu 1 \
    --concurrency 10 \
    --timeout 300 \
    --min-instances 0 \
    --max-instances 3
```

Add environment variables and any volume/persistence configuration required by your selected architecture. Do not put working credentials in source control.

Concurrency 10 is a coursework starting point — FastAPI + streaming handles concurrent requests, but per-user orchestration state and provider rate limits argue against blindly maximizing concurrency. Adjust based on your state-store design. Max instances may be capped during testing to prevent runaway cost.

4.6 Verification

After deploy prints the live URL:

- Load the URL in a browser and verify the application starts cleanly after a cold start.
- Submit a test query that exercises the RAG path — a query whose answer is in your corpus.
- Submit a test query that exercises the real-time knowledge path — a query about current information not contained in your static corpus.
- Verify the trace log shows the correct agent invoked, the correct LLM provider routed to, and the routing rationale.
- If your architecture claims durable multi-turn state, verify that state survives instance replacement/restart rather than merely surviving consecutive requests to one process.

---

5. Submission

Submit the following to Brightspace:

1. **Live deployment URL.** Cloud Run URL, publicly accessible, responds to arbitrary reasonable queries — not just to rehearsed test cases.

2. **Source code archive.** Zip of your repository, excluding `.env`, generated vector-index artifacts where appropriate, and any secrets. Include `requirements.txt` with pinned versions and `.env.example` with placeholders.

3. **Architecture documentation.** `docs/architecture.md` covering:

    - Diagram of the orchestration architecture — nodes/agents, communication paths, and relevant state schema (Mermaid or embedded image acceptable)
    - Agent responsibilities and tool access per agent
    - LLM routing table — which routing dimension drives each decision
    - RAG pipeline shape — corpus source, chunking strategy, retrieval `k`, augmentation prompt template
    - Durable-state design if your application claims state that survives Cloud Run instance replacement

4. **Evaluation report.** `docs/evaluation.md` covering:

    - RAGAS metrics — faithfulness, answer relevance, context precision, context recall — over your test question set
    - Latency measurements — per-query wall-clock time, broken down by retrieval, LLM call, and total. Report time-to-first-token (TTFT) and distinguish cold-start from warm latency: the first request to an idle Cloud Run service (min-instances=0) is much slower because the instance spins up and the

model/graph is "primed," so report warm (steady-state) numbers as your representative latency and note the cold-start figure separately. For reference, the reference deployment shows ~20–25 s TTFT on a cold start versus ~1.5–2 s warm.
    - Cost per query — measured against your actual API bills or estimated from token counts
    - Comparison across the LLM providers you routed to
5. **Process Log.** `PROCESS_LOG.md`, 200–300 words, four sections:
    - AI tools used (LLMs, versions where known)
    - Development narrative — what worked, what didn't, where you got stuck
    - Verification narrative — how you tested beyond running the code
    - What you learned — what the AI tools couldn't do that you had to figure out yourself
6. **README.** `README.md` covering setup, run instructions for local development, deploy instructions, provider requirements, and current cost/access information relevant to your implementation.

---

6. Academic integrity

This assignment is subject to USC's academic integrity policies. Specifically:

- **Individual work.** Assignment 2 is an individual assignment. You may discuss general concepts, framework choices, and orchestration patterns with classmates. Source code, deployment, and written deliverables must be your own work.
- **No sharing of source code** between students. This includes reference implementations (which are instructor-facing and not distributed during the assignment window) and prior semester deliverables.
- **AI-assisted development is allowed and expected.** This is a course about building AI systems. Using LLMs (Claude, GPT, Gemini, Grok, local models) to help write, debug, and refactor code is expected. Using LLMs to substitute for your own understanding of the material is not.
- **Multi-agent LLM instances.** Multi-agent systems involve multiple LLM instances communicating. Using different LLM providers for different agents is allowed and can reduce cost. Document the model/provider choice per agent in `docs/architecture.md`.
- **Frameworks and libraries.** Using LangGraph, LangChain, Chroma, Pinecone, RAGAS, FastAPI, MCP implementations, and the permitted Codex multi-agent path

is expected. These do not require special attribution beyond normal technical documentation.

- **Sanctions.** Violations are reported to the USC Office of Academic Integrity. Consequences range from assignment failure to course failure to expulsion, per USC policy.

---

7. Rubric

Base 100%. No bonus criteria. Weights ship verbatim from the syllabus.

7.1 Weight distribution

Criterion	Weight
Multi-agent orchestration correctness	25%
RAG implementation quality	20%
LLM routing logic	15%
Frontend usability	15%
Deployment and scalability	15%
A2A protocol integration or MCP tool sharing between agents	10%
Total	**100%**

7.2 Grading criteria detail

Multi-agent orchestration correctness — 25%

Full credit — LangGraph path: - LangGraph state graph compiles and executes end-to-end - Supervisor OR Hierarchical pattern implemented cleanly; three or more specialized agents with distinct tool access and distinct system prompts - Supervisor routes to workers based on inspectable state fields; failure of a worker node does not crash the graph - State schema documented in `docs/architecture.md`

Full credit — Codex worktree-based multi-agent path: - Three or more genuinely specialized agents execute through distinct delegated tasks/worktrees - Delegation and coordination are inspectable; the grader can identify which agent handled each task and how results were integrated - Agent roles are behaviorally distinct rather than cosmetic labels - Failure of one delegated agent/task is handled without collapsing the complete workflow - Worktree/task coordination architecture documented in `docs/architecture.md`

Partial credit: - Agents that are cosmetic labels on the same underlying LLM call with the same tool access - Orchestration exists but one or more transitions/delegations are hard-coded rather than state/task driven - Missing failure-mode handling in the orchestration layer

No credit: - No orchestration; single agent dressed as multi-agent - Selected orchestration mechanism does not execute

RAG implementation quality — 20%

Full credit: - Corpus ingested into Pinecone or Chroma with a documented chunking strategy - Retrieve → augment → generate pipeline implemented cleanly - Citations link generated answers back to source chunks - RAGAS metrics reported over a fixed test question set with concrete numbers

Partial credit: - RAG pipeline works but no RAGAS evaluation - Retrieval works but augmentation prompt doesn't preserve citations - Corpus is a five-file toy without genuine retrieval structure

No credit: - Keyword search dressed as RAG (no embeddings, no vector similarity) - No corpus ingestion pipeline

LLM routing logic — 15%

Full credit: - Two or more **different providers** wired into the system - Routing decision inspectable in the UI or trace log — grader can see which provider handled which query and why - Routing rationale documented (cost-based, capability-based, or hybrid)

Partial credit: - Routing works but the decision is opaque — grader has to read source code to understand which provider fires when - Router always picks the same provider under normal conditions - Multiple models are configured but they come from only one provider

No credit: - Single provider hard-coded across all agents - Routing configuration exists but is not exercised

Frontend usability — 15%

Full credit: - FastAPI backend serving SSE/token streams to a vanilla HTML/JS or permitted HTMX chat interface - Visible routing decisions per turn - Inspectable trace log showing orchestration transitions - Citations for RAG-retrieved content linked to source identifiers

Partial credit: - Functional UI but no visible routing or trace inspection - Trace log exists but is inscrutable - Response is delivered only after generation completes rather than streaming

No credit: - API-only deliverable with no frontend - Frontend that does not connect to the deployed backend

Deployment and scalability — 15%

Full credit: - Live Cloud Run URL responds to arbitrary reasonable queries - Deployment handles cold-start behavior without request failure - Environment variables and credentials are managed outside source code - README documents deploy commands that a TA can run against a fresh GCP project - If durable multi-turn state is claimed, an

external durable persistence mechanism such as Firestore, GCS/GCSFuse, Cloud SQL, or equivalent is implemented and documented

Partial credit: - Deployment responds only to the student's rehearsed queries - Cold-start behavior causes visible failures - Application claims durable multi-turn state but stores it only in process memory - Durable persistence is configured but incompletely documented

No credit: - Localhost-only submission (per syllabus §Assignment Submission Policy, localhost is not acceptable for Assignment 2) - Deployment URL returns errors or does not respond

> **Papa's rule:** A live deployment URL that only responds to your rehearsed test queries fails the deployment criterion. Deploy something that handles any reasonable query, not something that memorized your slides.

A2A protocol integration or MCP tool sharing between agents — 10%

Either path earns credit; pick one.

A2A protocol integration path: - Two or more agents communicate via A2A messages (Agent Card discovery, message/send, task lifecycle, SSE streaming) - A2A traffic inspectable in the trace log - Rationale for A2A choice documented in `docs/architecture.md`

MCP tool sharing path: - One or more MCP servers exposed as tools accessible to two or more agents - Same MCP tool invoked by different agents in different contexts - Tool-sharing rationale documented in `docs/architecture.md`

Partial credit: - One agent uses A2A or MCP; second agent does not participate - Integration exists but is not exercised by the deployed system

No credit: - No A2A messaging and no MCP tool sharing between agents

7.3 Late policy and regrade policy

Per syllabus. Grace days apply. Regrade requests via the Piazza form within the syllabus-specified window.